

ESPECIALIZACIÓN EN DESARROLLO DE SOFTWARE

Estándares y Métricas de Calidad en el Desarrollo de Software



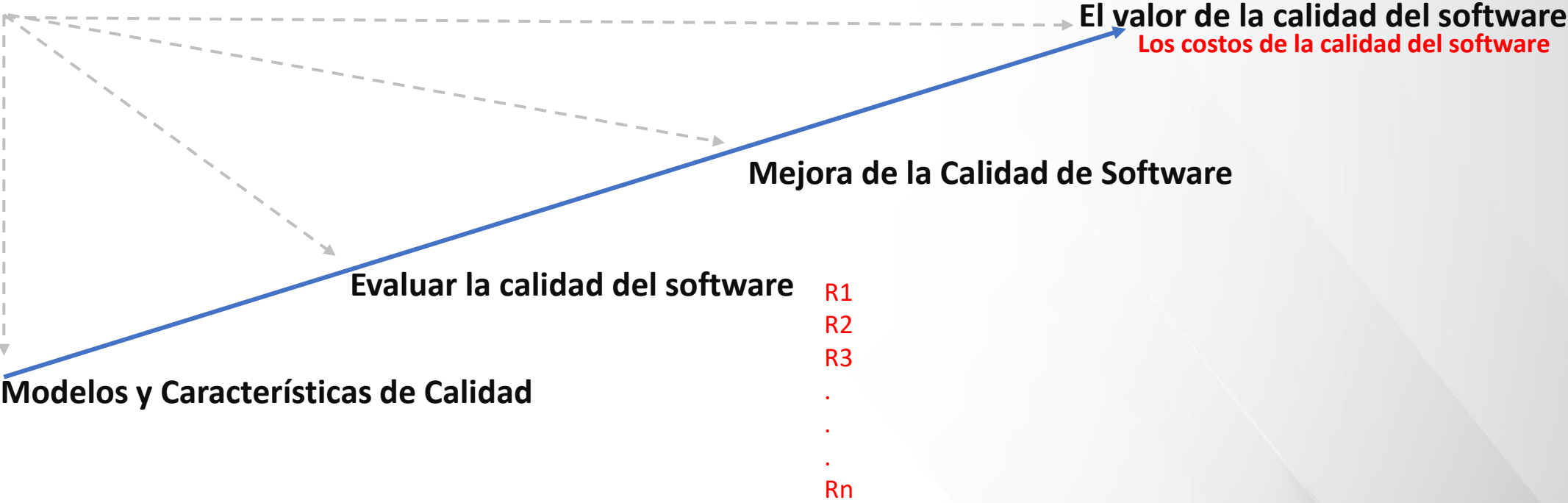
TEMARIO:

1. **FUNDAMENTOS DE LA CALIDAD DE SOFTWARE**
2. ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE: VALIDACIÓN, VERIFICACIÓN, REVISIONES Y AUDITORIAS
3. ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE: CARACTERIZACIÓN DE DEFECTOS
4. ESPECIFICACIÓN DE REQUISITOS DEL SOFTWARE
5. TÉCNICAS DE GESTIÓN DE CALIDAD
6. HERRAMIENTAS Y REQUISITOS DE CALIDAD DE SOFTWARE
7. TÉCNICAS DE GESTIÓN Y MEDICIÓN EN LA CALIDAD DE SOFTWARE
8. HERRAMIENTAS DE CALIDAD DE SOFTWARE

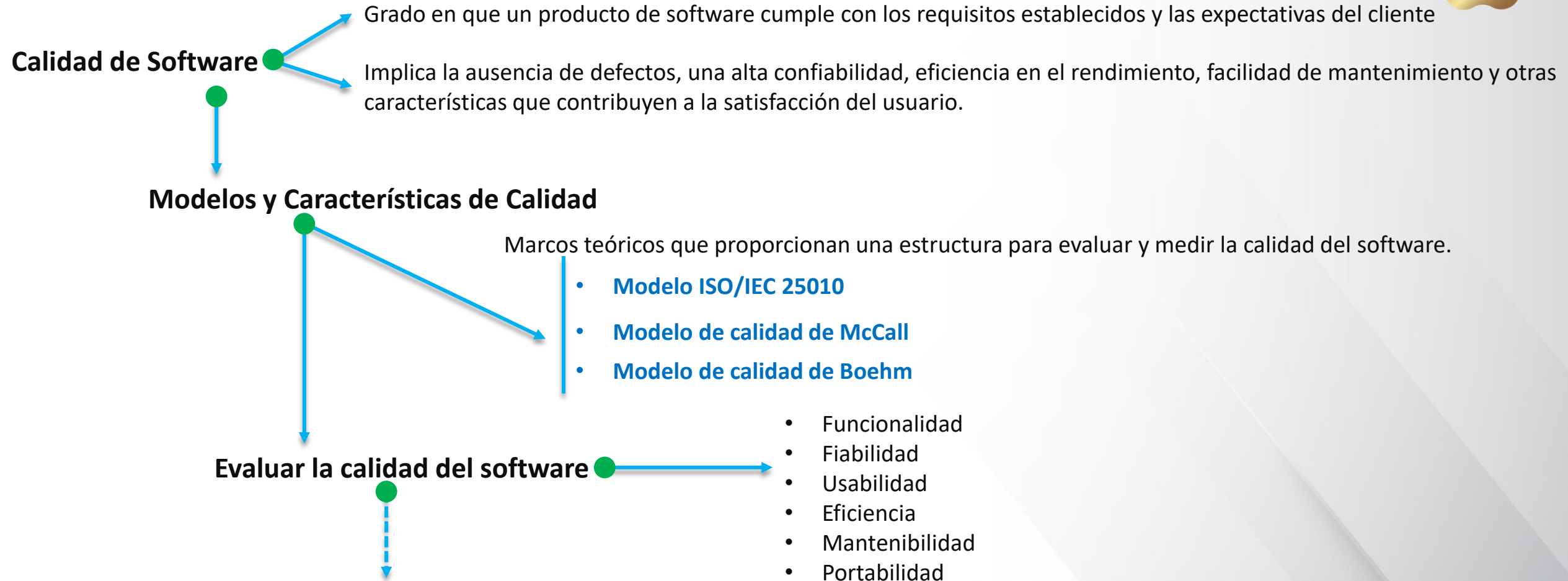


Fundamentos de la calidad de software: contexto

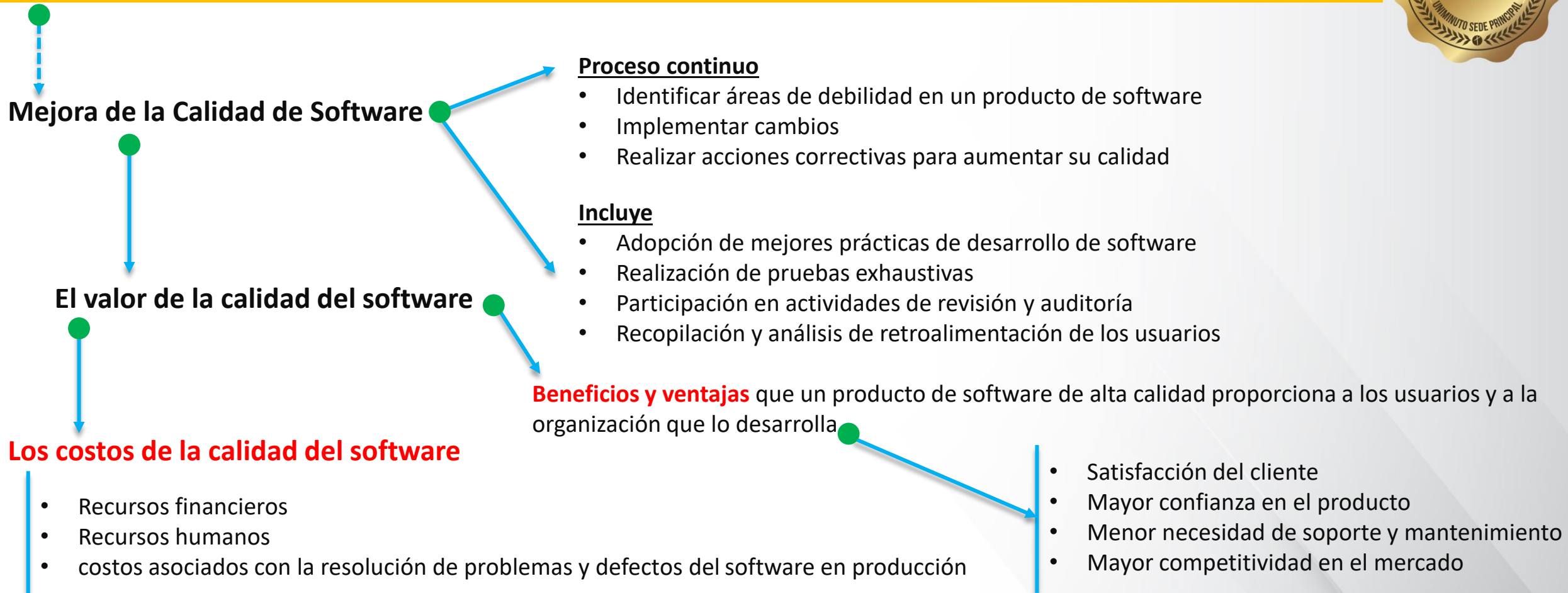
Calidad de Software



Fundamentos de la calidad de software: contexto



Fundamentos de la calidad de software: contexto



Fundamentos de la calidad de software: El camino a seguir

Fundamentos de la Calidad del Software	Descripción
Aseguramiento de la Calidad del Software (QA)	Conjunto de actividades planificadas y sistemáticas para proporcionar confianza en que el producto cumpla con los requisitos de calidad.
Validación y Verificación	Validación: asegurarse de que el software cumpla con los requisitos del usuario. Verificación: asegurarse de que el software se implemente correctamente.
Revisiones y Auditorías	Revisiones: examen formal de productos de trabajo para identificar defectos. Auditorías: evaluaciones sistemáticas para determinar el cumplimiento de estándares.
Caracterización de Defectos	Identificación, clasificación y documentación de defectos para comprender su naturaleza y tomar medidas correctivas.
Especificación de Requisitos del Software	Definición y documentación de requisitos del software para guiar el desarrollo y evaluación del sistema.
Técnicas de Gestión de Calidad	Prácticas y herramientas para planificar, controlar y mejorar la calidad del software.
Herramientas y Requisitos de Calidad de Software	Herramientas y técnicas para evaluar, monitorear y mejorar la calidad del software.
Técnicas de Gestión y Medición en la Calidad	Prácticas y procesos para gestionar y medir la calidad del software durante todo su ciclo de vida.
Herramientas de Calidad de Software	Herramientas específicas para apoyar las actividades de aseguramiento y gestión de la calidad del software.



Fundamentos de la calidad de software: Ejemplo

Fundamentos de la Calidad del Software	Ejemplo
Aseguramiento de la Calidad del Software (QA)	Implementación de un proceso de revisión de código antes de cada lanzamiento para garantizar que cumpla con los estándares de codificación.
Validación y Verificación	Pruebas de una nueva función en una aplicación móvil de comercio electrónico para asegurarse de que cumple con los requisitos del usuario.
Revisiones y Auditorías	Revisión del código, la arquitectura y la documentación de un proyecto antes de la entrega al cliente, seguido de una auditoría para garantizar cumplimiento.
Caracterización de Defectos	Documentación detallada de defectos encontrados durante las pruebas, incluyendo naturaleza, impacto y posibles causas para priorización y resolución.
Especificación de Requisitos del Software	Definición y documentación detallada de los requisitos de un software de contabilidad antes de iniciar el desarrollo.
Técnicas de Gestión de Calidad	Utilización de la metodología Scrum para gestionar el desarrollo de un software, incluyendo reuniones diarias, retrospectivas y tableros Kanban.
Herramientas y Requisitos de Calidad de Software	Uso de herramientas de prueba automatizadas como Selenium y JMeter para garantizar la calidad del software y reducir pruebas manuales.
Técnicas de Gestión y Medición en la Calidad	Definición de métricas de calidad como tasa de defectos por línea de código y satisfacción del cliente para medir y mejorar la calidad del software.
Herramientas de Calidad de Software	Utilización de herramientas de gestión de defectos como Jira o Bugzilla para registrar, priorizar y dar seguimiento a los problemas del software.



Caso de Estudio:

Desarrollo de una Aplicación Móvil de Entrega de Comida

Una empresa de tecnología está desarrollando una aplicación móvil de entrega de comida para competir en el mercado creciente de entrega de alimentos a domicilio.

El objetivo es crear una aplicación fácil de usar, confiable y eficiente que brinde una experiencia positiva a los usuarios y les permita realizar pedidos de comida de manera conveniente.



Fundamentos de la calidad de software: **Caso de estudio**

Caso de Estudio:

Desarrollo de una Aplicación Móvil de Entrega de Comida

Actividad	Resultado
Definición de Requisitos	
Diseño de la Interfaz de Usuario	
Desarrollo de la Aplicación	
Pruebas de Funcionalidad	
Pruebas de Rendimiento	
Lanzamiento y Despliegue	
Monitoreo y Soporte Post-Lanzamiento	



Fundamentos de la calidad de software: **Caso de estudio**

Caso de Estudio 2:

Desarrollo de una Aplicación Móvil de Entrega de Comida

Actividad	Resultado
Definición de Requisitos	Lista de requisitos establecida y documentada.
Diseño de la Interfaz de Usuario	Mockups y prototipos de la interfaz de usuario aprobados.
Desarrollo de la Aplicación	Aplicación móvil desarrollada de acuerdo con los requisitos y el diseño.
Pruebas de Funcionalidad	Todas las funciones de la aplicación pasaron las pruebas de funcionamiento.
Pruebas de Rendimiento	La aplicación mostró un rendimiento aceptable bajo diferentes cargas y condiciones.
Lanzamiento y Despliegue	La aplicación se lanzó con éxito en las tiendas de aplicaciones iOS y Android.
Monitoreo y Soporte Post-Lanzamiento	Problemas de rendimiento abordados y soporte técnico proporcionado según sea necesario.



Fundamentos de la calidad de software

Calidad del software → Software Quality Assurance (**SQA**) - Aseguramiento de la Calidad del Software (**ACS**)

- Conjunto de métodos, herramientas y técnicas utilizadas para gestionar y garantizar la calidad en el desarrollo de un producto de software
- El objetivo principal del SQA es asegurar que el software cumpla con los estándares de calidad definidos, los requisitos del cliente y las expectativas del usuario final.



Fundamentos de la calidad de software

Calidad del software —→ Software Quality Assurance (**SQA**) - Aseguramiento de la Calidad del Software (**ACS**)

Actividades comunes realizadas

Actividad	Descripción
Planificación de la calidad	Establecimiento de procesos, estándares y procedimientos para garantizar la calidad del software desde el inicio hasta el final del ciclo de vida del desarrollo.
Control de la calidad	Supervisión y seguimiento continuo del proceso de desarrollo para identificar y corregir posibles problemas de calidad a medida que surgen.
Revisión de requisitos y especificaciones	Verificación de que los requisitos del cliente y las especificaciones del software sean claros, completos y coherentes.
Pruebas de software	Implementación de pruebas exhaustivas para verificar la funcionalidad , la usabilidad , el rendimiento y la seguridad del software.
Gestión de la configuración	Control y seguimiento de los cambios realizados en el software y sus componentes para garantizar la integridad y la consistencia del producto final.
Auditorías de calidad	Evaluación periódica de los procesos de desarrollo y las prácticas de SQA para identificar áreas de mejora y asegurar el cumplimiento de los estándares de calidad.
Mejora continua	Identificación y aplicación de mejoras en los procesos de desarrollo y en las prácticas de SQA para optimizar la calidad del software y aumentar la eficiencia del equipo de desarrollo.



Fundamentos de la calidad de software

Integración del Software Quality Assurance (SQA) en las diferentes etapas del Software Development Life Cycle (SDLC)

Etapas del SDLC	Descripción	Actividades de SQA
Planificación	Establecimiento de objetivos y planificación del proyecto.	Definición de estándares de calidad y procesos de SQA.
Análisis	Recopilación y análisis de requisitos.	Revisión de requisitos para garantizar claridad y coherencia.
Diseño	Elaboración del diseño detallado del software.	Revisión de diseño y aseguramiento de cumplimiento de estándares de calidad.
Implementación	Desarrollo del software según el diseño establecido.	Supervisión del cumplimiento de estándares de codificación.
Pruebas	Realización de pruebas para identificar defectos.	Planificación, ejecución y seguimiento de pruebas de SQA.
Despliegue	Puesta en marcha del software en el entorno de producción.	Participación en pruebas de aceptación del usuario final.
Mantenimiento	Actualizaciones y mejoras continuas del software.	Gestión de calidad durante cambios y actualizaciones.



Fundamentos de la calidad de software

Integración del Software Quality Assurance (SQA) en las diferentes etapas del Secure Software Development Life Cycle (SSDLC)

Etapa del SSDLC	Descripción	Actividades de SQA
Planificación	Establecimiento de objetivos de seguridad y planificación del proyecto.	Definición de estándares de seguridad y procesos de aseguramiento de calidad.
Análisis de riesgos	Identificación y evaluación de posibles riesgos de seguridad.	Revisión de riesgos de seguridad y definición de controles de calidad.
Diseño seguro	Elaboración de un diseño seguro basado en los requisitos de seguridad.	Revisión de diseño seguro y aseguramiento de cumplimiento de estándares de seguridad.
Implementación segura	Desarrollo del software con enfoque en prácticas seguras de codificación.	Supervisión del cumplimiento de estándares de codificación segura.
Pruebas de seguridad	Realización de pruebas para identificar vulnerabilidades y fallos de seguridad.	Planificación, ejecución y seguimiento de pruebas de seguridad.
Despliegue seguro	Implementación segura del software en el entorno de producción.	Participación en pruebas de seguridad de aceptación del usuario final.
Mantenimiento seguro	Actualizaciones y mejoras continuas del software, manteniendo la seguridad.	Gestión de calidad durante cambios y actualizaciones de seguridad.

Fundamentos de la calidad de software

Las métricas

- Es una **medida cuantitativa o cualitativa** utilizada para evaluar diversos aspectos del proceso de desarrollo, calidad y desempeño del software.
- **Proporcionan datos objetivos que permiten monitorear, controlar y mejorar el proceso** de desarrollo de software, así como el producto resultante.
- Pueden **abarcар una amplia variedad de áreas**, como el **tiempo**, el **costo**, la **calidad**, la **seguridad** y el **rendimiento**.
- **Son herramientas importantes en la gestión de proyectos de software y en el aseguramiento de la calidad**, ya que proporcionan una base objetiva para la toma de decisiones y la evaluación del progreso del proyecto



Fundamentos de la calidad de software

Las métricas

Métrica	Descripción
Cobertura de código	Porcentaje del código fuente que ha sido ejecutado durante las pruebas.
Número de defectos por línea de código	Cantidad de defectos encontrados en relación con la cantidad de líneas de código escritas.
Tiempo de resolución de defectos	Tiempo promedio necesario para corregir un defecto encontrado durante el desarrollo.
Velocidad de entrega	Cantidad de trabajo completado por unidad de tiempo, como historias de usuario entregadas por semana o mes.
Satisfacción del cliente	Evaluación del cliente sobre la calidad y el rendimiento del software.
Nivel de riesgo	Probabilidad e impacto de los riesgos identificados en el proyecto de desarrollo de software.



Fundamentos de la calidad de software: **Caso de estudio 2**

Caso de Estudio: Desarrollo de una Aplicación de Comercio Electrónico

Una empresa de comercio electrónico está desarrollando una nueva plataforma en línea para vender productos electrónicos. El objetivo es crear una plataforma segura, rápida y fácil de usar que proporcione una experiencia de compra satisfactoria para los clientes y aumente las ventas de la empresa.



Fundamentos de la calidad de software: **Caso de estudio 2**

Caso de Estudio 2: Desarrollo de una Aplicación de Comercio Electrónico

Métricas para Evaluar el Caso de Estudio

Métrica	Descripción
Tiempo de desarrollo	Tiempo total requerido para completar el desarrollo de la plataforma de comercio electrónico.
Costo del proyecto	Costo total estimado para desarrollar la plataforma, incluyendo recursos humanos y materiales.
Velocidad de carga	Tiempo promedio que tarda la página principal en cargar en el navegador del usuario.
Tasa de conversión	Porcentaje de visitantes del sitio web que realizan una compra en comparación con el total de visitantes.
Número de transacciones	Cantidad total de transacciones realizadas en la plataforma durante un período de tiempo determinado.
Índice de abandono del carrito	Porcentaje de usuarios que abandonan el proceso de compra después de agregar productos al carrito de compras



Fundamentos de la calidad de software: **Caso de estudio 2**

Caso de Estudio 2: Desarrollo de una Aplicación de Comercio Electrónico

Métricas para Evaluar el Caso de Estudio

¿Otra métrica?

Métrica	Descripción



Fundamentos de la calidad de software

Cultura y ética de la ingeniería de software

Aspecto	Descripción
Colaboración y Trabajo en Equipo	Promoción de la comunicación abierta, el respeto mutuo y la capacidad de trabajar en equipo para lograr objetivos comunes. Fomento de la colaboración entre diferentes roles y disciplinas dentro de los equipos de desarrollo de software.
Aprendizaje Continuo	Valorización del aprendizaje continuo para mantenerse actualizado con las últimas tendencias y avances en tecnología. Fomento de la adquisición de nuevas habilidades y conocimientos a lo largo de la carrera profesional.
Flexibilidad y Adaptabilidad	Promoción de la capacidad de adaptación a cambios rápidos y frecuentes en el entorno tecnológico. Fomento de la apertura a nuevas ideas, métodos y herramientas, así como afrontar desafíos con creatividad y resiliencia.
Énfasis en la Calidad	Valorización de la calidad del software mediante prácticas como la revisión de código, las pruebas exhaustivas y la adopción de estándares y mejores prácticas. Compromiso con el desarrollo de software confiable, seguro y eficiente que satisfaga las necesidades y expectativas de los usuarios.
Responsabilidad Social	Reconocimiento de la responsabilidad ética de considerar el impacto social de las decisiones y acciones en el desarrollo de software. Compromiso con el desarrollo de tecnologías que respeten los derechos humanos, promuevan la inclusión y la equidad, y minimicen los posibles efectos negativos en la sociedad.
Integridad Profesional	Mantenimiento de altos estándares de integridad y honestidad en todas las actividades profesionales. Compromiso con la transparencia en la comunicación, el respeto a la propiedad intelectual y la evitación de prácticas engañosas o fraudulentas.
Privacidad y Seguridad de los Datos	Responsabilidad de proteger la privacidad y seguridad de los datos de los usuarios. Compromiso con el diseño de sistemas con medidas adecuadas de seguridad cibernética y el respeto a la confidencialidad de la información personal.
Desarrollo Ético de la Inteligencia Artificial	Consideración de los posibles sesgos, discriminaciones y riesgos éticos asociados con los sistemas de inteligencia artificial desarrollados. Compromiso con la transparencia en los algoritmos y la implementación de salvaguardias para evitar consecuencias no deseadas, promoviendo un desarrollo ético y responsable de la IA.

Fundamentos de la calidad de software: **Caso de estudio 3**

Cultura y ética de la ingeniería de software

Caso de Estudio: Desarrollo de una Aplicación Móvil de Salud

Una empresa de tecnología está desarrollando una aplicación móvil de salud destinada a ayudar a las personas a llevar un estilo de vida más saludable. La aplicación proporcionará funciones como seguimiento de actividad física, registro de dieta y nutrición, monitoreo de signos vitales y recordatorios para medicamentos y citas médicas.

Visión holística del enfoque ético y cultural en el desarrollo de software





Fundamentos de la calidad de software: **Caso de estudio 3**

Cultura y ética de la ingeniería de software

Caso de Estudio: Desarrollo de una Aplicación Móvil de Salud

Aspecto	Descripción
Colaboración y Trabajo en Equipo	El equipo de desarrollo trabaja colaborativamente, con comunicación abierta y respetuosa entre diseñadores, desarrolladores y profesionales de la salud. Se valoran las opiniones de cada miembro del equipo para crear una aplicación integral.
Aprendizaje Continuo	Se mantienen actualizados con las últimas investigaciones y avances en salud y tecnología, participando en cursos y eventos relevantes. Se incorporan constantemente nuevas funcionalidades basadas en la retroalimentación del usuario y la medicina preventiva.
Énfasis en la Calidad	Se realizan pruebas exhaustivas para garantizar un funcionamiento seguro y confiable. Se siguen las mejores prácticas de diseño para una experiencia de usuario intuitiva y accesible.
Responsabilidad Social	Se toman medidas para proteger la privacidad y seguridad de los datos de los usuarios, obteniendo su consentimiento informado y aplicando medidas de seguridad cibernética.
Integridad Profesional	Se adhieren a altos estándares de integridad y ética profesional, evitando conflictos de intereses y garantizando la transparencia en la comunicación con los usuarios.
Desarrollo Ético de la IA	Se presta atención a posibles sesgos y riesgos éticos en el uso de algoritmos de IA, implementando salvaguardias para garantizar la equidad y la imparcialidad en el procesamiento de datos.



Fundamentos de la calidad de software

Modelos – Estándares y características de calidad

Estándar	Descripción	Función
CMM (Capability Maturity Model)	Modelo utilizado para evaluar y mejorar la madurez de los procesos de desarrollo de software en una organización.	Sirve como un marco de referencia para comprender y mejorar la capacidad de una organización para desarrollar software de alta calidad de manera consistente y predecible.
ISO/IEC 15504 (SPICE)	Estándar internacional utilizado para evaluar y mejorar la capacidad de los procesos de desarrollo y mantenimiento de software en una organización.	Proporciona un enfoque estructurado y sistemático para la evaluación y mejora de los procesos de software, ayudando a las organizaciones a aumentar su eficiencia y calidad en el desarrollo de software.
CMMI (Capability Maturity Model Integration)	Evolución del CMM que integra varios modelos de madurez de capacidad. Sirve como un marco integral para mejorar y evaluar la madurez de los procesos en diferentes áreas de una organización.	Permite establecer prácticas de gestión y desarrollo de software efectivas y eficientes en toda la empresa, resultando en una mayor eficiencia, calidad y consistencia en la entrega de productos de software.
ISO/IEC 25010	Estándar que define un modelo de calidad de producto de software, especificando un conjunto de características y subcaracterísticas de calidad.	Proporciona un marco para definir, medir y mejorar la calidad del producto de software en términos de sus características, rendimiento, fiabilidad, usabilidad, seguridad y otros atributos importantes.
Modelo de Calidad de McCall	Modelo que define seis áreas de calidad de software: corrección, fiabilidad, eficiencia, integridad, usabilidad y adaptabilidad.	Proporciona un marco para evaluar la calidad del software en términos de diversas áreas que afectan su funcionalidad, fiabilidad, rendimiento y capacidad de adaptación.
Modelo de Calidad de Boehm	Modelo que se centra en la calidad del producto y del proceso. Define áreas de calidad de producto, como fiabilidad, eficiencia y usabilidad, y calidad del proceso, como eficacia y eficiencia.	Proporciona un marco para evaluar y mejorar tanto la calidad del producto de software como la eficacia y eficiencia de los procesos utilizados para desarrollarlo.

Fundamentos de la calidad de software

Modelos y características de calidad

Son marcos teóricos y atributos específicos utilizados para evaluar y medir la calidad del software.

Funcionalidad: Capacidad del software para proporcionar funciones que satisfagan las necesidades especificadas.

Fiabilidad: Capacidad del software para mantener un nivel de rendimiento especificado bajo condiciones establecidas durante un período de tiempo determinado.

Modelo ISO/IEC 25010

Usabilidad: Facilidad de uso del software por parte de los usuarios previstos para realizar tareas específicas con efectividad, eficiencia y satisfacción.

Eficiencia en el desempeño: Desempeño relativo del software en términos de cantidad de recursos utilizados en relación con la cantidad de recursos proporcionados.

Compatibilidad: Capacidad del software para interactuar con otros sistemas especificados.

Mantenibilidad: Capacidad del software para ser modificado, incluyendo correcciones, mejoras o adaptaciones.

Portabilidad: Capacidad del software para ser transferido de un entorno a otro.

Fundamentos de la calidad de software

Modelos y características de calidad

Son marcos teóricos y atributos específicos utilizados para evaluar y medir la calidad del software.

Corrección: Capacidad del software para realizar sus funciones requeridas sin fallas.

Fiabilidad: Probabilidad de que el software funcione sin fallas durante un período específico y bajo ciertas condiciones.

Modelo de Calidad de McCall

Eficiencia: Capacidad del software para realizar sus funciones requeridas dentro de los límites de recursos establecidos.

Integridad: Capacidad del software para proteger contra el acceso no autorizado o el daño.

Usabilidad: Facilidad de uso y comprensión del software.

Adaptabilidad: Capacidad del software para ser modificado para satisfacer diferentes requisitos de usuario o entorno.

Fundamentos de la calidad de software

Modelos y características de calidad

Son marcos teóricos y atributos específicos utilizados para evaluar y medir la calidad del software.

Modelo de Calidad de Boehm

Calidad de Producto:

Incluye características como fiabilidad, eficiencia, usabilidad y mantenibilidad.

Calidad del Proceso:

Incluye características del proceso de desarrollo de software, como la eficacia, la eficiencia y la adherencia a los estándares y procedimientos establecidos.



Fundamentos de la calidad de software

Modelos y características de calidad

Modelo de Calidad	Características	Ejemplo
ISO/IEC 25010	Funcionalidad, Fiabilidad, Usabilidad, Eficiencia en el desempeño, Compatibilidad, Mantenibilidad, Portabilidad	Un sistema de gestión de ventas en línea debe tener funcionalidades que permitan a los usuarios realizar compras, agregar productos al carrito y realizar pagos de forma segura. Además, debe ser confiable para manejar grandes volúmenes de transacciones y estar disponible durante horas pico de tráfico. La interfaz de usuario debe ser intuitiva y fácil de usar, permitiendo a los usuarios navegar por el sitio y completar transacciones sin dificultad. El sistema debe ser eficiente en términos de tiempo de carga de páginas y procesamiento de pedidos. También debe ser compatible con diferentes navegadores web y dispositivos, y ser fácilmente mantenible y adaptable a cambios futuros.



Fundamentos de la calidad de software

Modelos y características de calidad

Modelo de Calidad	Características	Ejemplo
Modelo de Calidad de McCall	Corrección, Fiabilidad, Eficiencia, Integridad, Usabilidad, Adaptabilidad	Un sistema de gestión de inventario debe ser correcto en términos de mantener registros precisos de los niveles de inventario y evitar discrepancias. Debe ser fiable para garantizar que los datos de inventario estén disponibles cuando sea necesario y no se pierdan ni se corrompan. Además, debe ser eficiente en el uso de recursos de hardware y software, minimizando el tiempo de procesamiento de transacciones y consultas. La integridad del sistema es fundamental para proteger los datos de inventario contra accesos no autorizados o modificaciones indebidas. La usabilidad es importante para que los usuarios puedan realizar fácilmente tareas como agregar, eliminar o actualizar elementos de inventario. Finalmente, el sistema debe ser adaptable para permitir la incorporación de nuevas funcionalidades o cambios en los requisitos del negocio.



Fundamentos de la calidad de software

Modelos y características de calidad

Modelo de Calidad	Características	Ejemplo
Modelo de Calidad de Boehm	Calidad de Producto, Calidad del Proceso	La calidad de producto se refiere a las características del software, como la fiabilidad, la eficiencia, la usabilidad y la mantenibilidad. Por ejemplo, un sistema de reservas de vuelos debe ser confiable para que los usuarios puedan reservar vuelos sin preocuparse por errores o fallos. También debe ser eficiente en términos de velocidad de búsqueda y reserva de vuelos. La interfaz de usuario debe ser fácil de usar para que los usuarios puedan navegar y reservar vuelos sin dificultad. Además, el sistema debe ser fácilmente mantenible para permitir correcciones de errores y actualizaciones periódicas. La calidad del proceso se refiere a la eficacia y la eficiencia del proceso de desarrollo de software. Por ejemplo, un equipo de desarrollo que sigue prácticas ágiles puede ser más efectivo en la entrega de software de alta calidad de manera oportuna y dentro del presupuesto.

Fundamentos de la calidad de software: **Caso de estudio 4**

Modelos y características de calidad

Una tienda en línea está desarrollando un nuevo sistema de gestión de ventas para mejorar la eficiencia y la experiencia del usuario.

El sistema debe manejar el proceso de ventas desde la selección de productos hasta el pago y la entrega.

Se necesita una evaluación exhaustiva de la calidad del software para garantizar que el sistema cumpla con los estándares de calidad esperados y proporcione una experiencia satisfactoria para los usuarios.



Fundamentos de la calidad de software: **Caso de estudio 4**

Modelos y características de calidad

Aspecto de Calidad	Métrica	Descripción
Funcionalidad	Cobertura de Pruebas	Porcentaje de código o funcionalidad cubierta por pruebas automatizadas.
	Complejidad Ciclomática	Número de caminos linealmente independientes a través del código.
Fiabilidad	Tasa de Fallas	Número de fallas experimentadas durante un periodo específico.
	Tiempo Medio entre Fallas (MTBF)	Promedio de tiempo entre fallos en el software.
Usabilidad	Tiempo de Aprendizaje	Tiempo requerido para que un usuario aprenda a utilizar el software.
	Tasa de Error de Usuario	Número de errores cometidos por los usuarios al interactuar con el software.
Eficiencia	Tiempo de Respuesta	Tiempo promedio que tarda el sistema en responder a las solicitudes.
	Uso de Recursos	Utilización de recursos como CPU, memoria y almacenamiento.
Mantenibilidad	Índice de Cohesión	Grado en que los componentes de software están relacionados entre sí.
	Índice de Acoplamiento	Grado en que los componentes de software dependen entre sí.
Portabilidad	Adaptabilidad	Facilidad con la que el software puede ser adaptado a diferentes entornos.
	Reusabilidad	Porcentaje de componentes de software que pueden ser reutilizados.